iPRA: coverage before cancellation, standard-numeral rigidity, and smaller proof kernels

Research prepared for Vladimir Reshetnikov
with ChatGPT

10 November 2026

Abstract

This investigation produces a new coverage theorem for the scalar self-seeded arithmetic in the supplied drafts. Without assuming successor injectivity, the weak theory proves that every element is zero or a successor, that successor has no fixed points, and that every fixed standard positive numeral has exactly its intended predecessor. With injectivity retained, the predecessor construction needs only two exceptional inputs and no joint coverage/predecessor induction. A separate logical theorem identifies one native paired binder over an equation with the explicitly prenex existential-universal quantifier-free class, explaining precisely why weakening the induction guard is substantive.

Complete proof checkers for the inherited initialized-iterator presentation are also reduced: the recommended explicit-list program has 539 normalized Python bytes; a compact equivalent has 531. A distinct generic profile has 289 AST nodes. The current comparison envelope is 542 bytes, 261 tokens, and 291 AST nodes, attained by different programs. No new program beats all three coordinates, and the token minimum is unchanged. The arithmetic lemma is not assigned the sizes of these inherited-arithmetic checkers.

Unresolved target. This is not a proof of an absolutely simplest axiomatization or the requested significant complete-checker breakthrough. Successor injectivity in the weak self-seeded theory remains unproved here; neither its full iPRA bi-interpretability nor a countermodel is claimed. The archive contains complete sources, attributed equivalence proofs, executable certificates, measurements, and negative controls.

Contents

1 Result, comparison, and scope **3**

2 The complete calculus implemented by the measured checkers **4**

3 Coverage before cancellation in the scalar self-seeded basis **5**

4 The exact logical strength visible at one native binder **8**

5 The complete proof kernels and their invariants **11**

6 Evidence, reproducibility, and limitations **14**

7 Further research questions **15**

A The ordinary intuitionistic logic used in the arithmetic arguments **16**

B Reconstructing the intuitionistic logical interface **17**

C Internal scalar predecessor bootstrap **20**

D From literal-equation induction to full intuitionistic PRA **24**

E Pairing, primitive recursion, and internal numerical composites **29**

F Arithmetic strength and both ordinary interpretation composites **31**

G Attribution of the raw-input audit **33**

H Why the absent major can be a proved empty array **33**

1 Result, comparison, and scope

The audit freezes the 20 direct ZIP archives in `VladimirReshetnikov/ProveIt`, under `docs/drafts`, at this commit:[1]

`b9c4d74a0018bdafc5bc06e755a0145f4ed2b8d6`

The static audit checks each archive's Git blob identity and length, recurses through 35 distinct ZIP contents, and measures 484 distinct Python source byte strings. There are 2,744 source occurrences under the deduplicated-archive traversal. At or below any previously reported candidate threshold, 48 distinct source files require classification: 10 complete theorem checkers and 38 support programs. The latter are build scripts, test runners, translators, measurers, or integrity tools. They cannot be counted as standalone theorem checkers.

A final read-only check of commit `89f5bdfe` found that the archives had been curated into `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research`. All 2,769 retained research files match original archive members by Git blob identity, including all 1,605 Python files and all 187 TeX files. Only the project README and attributes file are new. All three comparison-frontier programs remain present and unchanged; the migration does not alter this comparison.[2] The exact hashes and remeasurements are recorded in `evidence/latest_snapshot_recheck.json`.

The observed coordinate minima in both inspected snapshots are

$$(\text{bytes}, \text{tokens}, \text{AST nodes}) = (542, 261, 291).$$

The first two occur in different neutral-major profiles; the third occurs in the earlier lexical-loop profile.[3, 4] They are observed minima within this fixed corpus, hence upper bounds on an unknown optimum. They are not mathematical lower bounds on all possible theories or implementations.

Whole standalone module	Bytes	Tokens	AST	Gzip-9
Current byte leader	542	263	309	343
Current token leader	578	261	295	346
Current AST leader	642	263	291	353
New explicit-list checker	539	266	308	340
New compact-list checker	531	266	310	335
New generic-loop checker	613	263	289	349

The byte improvement is $11/542 \approx 2.03\%$ for the compact profile. The AST improvement is $2/291 \approx 0.69\%$ for a different program. The explicit-list program is recommended for review and ordinary use: its parsing recursion follows finite JSON trees. The generic profile has a malformed-string divergence qualification under an ideal unlimited stack, explained in Section 5. These programs implement the same initialized-iterator arithmetic. Their size reductions compose existing neutral-major, lexical-scope, and control-flow techniques; they are not presented as new arithmetic axioms.

There are three logically separate contributions:

1. The new theorem about the weaker self-seeded arithmetic in Section 3; it does not complete an injectivity-free iPRA foundation.
2. The binder-depth expressivity calculation in Section 4; it is a precise induction-guard diagnostic, not a universal checker-size lower bound.

3. Smaller complete presentations of the inherited, fully iPRA-equivalent calculus in Sections 2 and 5; these satisfy full-domain bi-interpretability through the included inherited bridge.

2 The complete calculus implemented by the measured checkers

This section explicitly fixes every primitive numerical schema and inference pattern. The construction is the inherited Choice-DI calculus in formula-context form, with a new composition of certificate techniques. The complete mathematical theory must not be confused with the weaker self-seeded theory investigated in the next section.

2.1 Numerical and formula syntax

There is one numerical sort. Terms are

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only u in h is bound by the iterator. Every update is fixed finite earlier-stage term syntax; all other variables are passive numerical parameters. Equivalently, introduce one first-order symbol I_h for each such finite description. There are no numerical function codes and no uniformly total evaluator.

The two computation schemas are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

Native formulas are

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

The designated absurdity is $0 = S0$. The intended ordinary readings are

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ(y), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

These are interpretation clauses, not additional primitive connectives. The Q binder is conservative shorthand for unused-witness D; its explicit presence makes the checker interface smaller.

2.2 All primitive inference patterns

Contexts consist of reusable native formulas. Substitutions avoid capture. The complete rule inventory, in addition to I0 and IS, is:

1. **Hypothesis.** Infer $\Gamma \vdash A$ when $A \in \Gamma$.
2. **Transport.** From $\Gamma \vdash s = t$ and $\Gamma \vdash H(s)$, infer $\Gamma \vdash H(t)$, for any fixed native formula template H.
3. **Closure elimination.** From $\Gamma \vdash \mathsf{Q}_y B(y)$ and $\Gamma \vdash B(0)$, infer $\Gamma \vdash B(t)$.
4. **Paired elimination.** From $\Gamma \vdash \mathsf{D}_{x,y} B(x, y)$ and $\Gamma, \mathsf{Q}_y B(a, y) \vdash A$, infer $\Gamma \vdash A$. The parameter a is fresh for Γ, A and the passive matrix data. The displayed local assumption is discharged.
5. **Case introduction.** From $\Gamma, B(t, 0) \vdash B(t, Sz)$, infer $\Gamma \vdash \mathsf{D}_{x,y} B(x, y)$.

6. **Equation-progress introduction.** From $\Gamma, E(t,z) \vdash E(t,Sz)$, infer $\Gamma \vdash \mathsf{D}_{x,y}E(x,y)$, only when E is a literal numerical equation. An equation below a binder is not a literal equation.

In the last two rules z is fresh for the retained context, witness, and fixed matrix parameters; the displayed assumption is discharged. Weakening, exchange, contraction, renaming, cut, and numerical substitution are admissible by induction on finite derivations. No generalization rule allowing a retained parameter to change during an induction is used.

No further numerical or logical axioms are primitive here. Reflexivity, successor injectivity, numerical coverage, the ordinary intuitionistic connectives, and full quantifier-free induction are derived. The appendices supply the construction in a noncircular order.

2.3 Why this is fully equivalent to iPRA

The equation-progress rule, an unused witness in D, and its elimination derive contextual literal-equation induction. Case introduction similarly gives numerical cases. Computation and transport give reflexivity: if $r = I(0,t;u.0)$, use $r = t$ as both transport premises with the motive $v = t$ to obtain $t = t$.

The derived selector $C(v,a,b) = I(v,a;u.b)$ permits formula gating and selection. Appendix B explicitly reconstructs intuitionistic implication, conjunction, disjunction, both quantifiers, and falsity, including both formula round trips. The scalar predecessor is constructed using only equation induction and numerical cases in Appendix C. Equality reflection then yields quantifier-free induction; pairing compiles every fixed general PR description. Internal recurrence uniqueness identifies both numerical return translations. The isomorphism of the original domain with its double interpretation has graph $x = y$ in both theories.

Thus the theory implemented by each selected checker is equiconsistent and bi-interpretable with full first-order iPRA. This statement concerns all native formulas and retained contexts, including quantified ones, and the entire numerical domain. It is stronger than equality of standard numerical functions or equiconsistency alone. The underlying mathematical bridge is inherited and explicitly attributed; the new checker composition does not change it.

3 Coverage before cancellation in the scalar self-seeded basis

This section isolates a mathematical simplification of the supplied scalar self-seeded presentation. It is not a complete new basis below the measured checker frontier. In particular, it does not justify deleting successor injectivity from that presentation.

Use ordinary intuitionistic first-order logic with equality, terms $0, S, \epsilon$, and a staged scalar recursor $F(n;v.h)$ with equations

$$F(0;v.h) = S(h[0/v]), \qquad F(Sn;v.h) = h[F(n;v.h)/v].$$

The remaining nonlogical assumptions, besides contextual induction on literal equations, are explicitly

$$0 = S0 \to \bot, \qquad \epsilon(a,b) = 0 \vee \epsilon(a,b) = 1, \qquad \epsilon(a,b) = 1 \leftrightarrow a = b.$$

All free numerical variables in these schemas are universally closed. Here $1 = S0$, and each recursor body is an earlier-stage source term. Neither successor injectivity nor numerical coverage is assumed in the result below.

Put $D(c,a) = F(c; v.a)$ and $Z(c) = D(c,0)$. Thus

$$D(0,a) = Sa, \quad D(Sn,a) = a, \quad Z(0) = 1, \quad Z(Sn) = 0.$$

If $Sn = 0$, these equations give $0 = 1$, so every successor is nonzero. For Boolean b, c, the source terms

$$\neg_B b = \epsilon(b,0), \qquad b \wedge_B c = \epsilon(D(b, \epsilon(c,0)),0)$$

have their usual truth tables; define $\vee_B$ by De Morgan. Only $0 \neq 1$ and successor nonzeroness are used in these finite cases. The usual external syntax recursion assigns a Boolean characteristic to each fixed quantifier-free formula. Literal-equation induction therefore gives contextual quantifier-free induction, without successor cancellation.

Define the following fixed source terms:

$$\begin{aligned} B_a(v) &= \epsilon(v,0) \vee_B \epsilon(Sv,a), \\ H_a(v) &= D(B_a(v), v), \\ U(j,a) &= F(j; v.H_a(v)), \qquad p_*(a) = U(a,a). \end{aligned}$$

The update holds the state at zero or immediately before a, and increments it in the remaining case. Since $B_a(0) = 1$, $H_a(0) = 0$ and $U(0,a) = 1$. With a fixed, quantifier-free induction gives

$$U(j,a) = Sj \quad \vee \quad S(U(j,a)) = a. \tag{1}$$

At zero the first alternative holds. A state satisfying the second alternative is fixed by H_a. Otherwise the first alternative advances from Sj to $S(Sj)$: in that branch the current state is a successor and hence nonzero. No separate induction asserting nonzeroness of the whole orbit is needed.

After this induction has been proved with its parameter fixed, specialize $j = m$ and $a = S^2 m$. Both alternatives of (1) imply $S(U(m, S^2 m)) = S^2 m$. The state is then fixed. Exactly two further computation steps give

$$S(p_*(S^2 m)) = S^2 m. \tag{2}$$

This is a right-inverse equation only; no cancellation of S has taken place.

Whole-domain coverage theorem. The theory with the assumptions just listed proves

$$n = 0 \ \vee \ \exists m \, n = Sm.$$

Indeed, apply quantifier-free induction to

$$C(n): \quad n = 0 \ \vee \ n = 1 \ \vee \ S(p_*(n)) = n.$$

The zero case is immediate. In the successor step, $n = 0$ gives $Sn = 1$. If $n = 1$, apply (2) with $m = 0$. In the third case $n = S(p_*(n))$, whence $Sn = S^2(p_*(n))$; apply (2) with $m = p_*(n)$ and rewrite. This proves $C(n)$ for every n. Its second and third alternatives provide the witnesses 0 and $p_*(n)$ respectively. The proof covers nonstandard elements and never assumes that an arbitrary element is reached by a standard finite successor iteration.

3.1 No fixed points and rigidity of each standard numeral

The same weak theory proves more than coverage, although the following results still fall short of arbitrary successor cancellation.

Successor has no fixed point. Let $h(v) = \epsilon(v, 0)$. This source update has no fixed point. Indeed, $h(v) = v$ and the Boolean range of h give $v = 0$ or $v = 1$; the two cases contradict $h(0) = 1$ and $h(1) = 0$, respectively. If $Sa = a$, the recursor equation would give

$$F(a; v.h) = F(Sa; v.h) = h(F(a; v.h)),$$

contrary to that fact. Thus $Sa \neq a$ for every a, without using successor injectivity or coverage. In particular, $1 \neq 2$.

The only predecessor of one is zero. Put

$$h(v) = \neg_B \epsilon(v, 2), \qquad A(n) = F(n; v.h).$$

The distinctness of $0, 1, 2$ gives $h(0) = 1$, $A(0) = 2$, and $A(1) = h(2) = 0$. The image of h is contained in $\{0, 1\}$, and

$$h(v) = 0 \quad \longleftrightarrow \quad v = 2.$$

Quantifier-free induction proves $A(n) = 2 \leftrightarrow n = 0$. The zero case holds, whereas at every successor both sides are false: $A(Sn) = h(A(n))$ is zero or one, and $Sn \neq 0$. If $Sn = 1$, then $h(A(n)) = A(Sn) = A(1) = 0$; hence $A(n) = 2$ and $n = 0$. The converse is immediate. Therefore

$$Sn = 1 \quad \longleftrightarrow \quad n = 0.$$

Finite-prefix rigidity metatheorem. For every fixed standard integer $k \geq 1$, there is a finite derivation in the weak theory of

$$Sx = \overline{k} \quad \longleftrightarrow \quad x = \overline{k-1}. \tag{3}$$

Here bars denote fixed numerals. This is an external metatheorem, not a derivation of a single formula quantifying over k.

The proof is an external induction on k. Carry as induction data the distinctness of the numerals $0, \ldots, k+1$ and (3) for all smaller positive indices. The initial distinctness of $0, 1, 2$ follows above. Define the following finite numeral-valued source update:

$$h_k(v) = \begin{cases} \overline{i-1}, & v = \bar{i}, \quad 1 \leq i \leq k-1, \\ \overline{k-1}, & v = \overline{k+1}, \\ \overline{k}, & \text{otherwise.} \end{cases}$$

This display abbreviates an actual source term, not an additional recursion schema over a graph-defined function. To construct it, for each fixed $r = 1, \ldots, k$ form the Boolean characteristic $b_r(v)$ of the finite condition that the displayed value is at least r. Explicitly, b_r is the Boolean negation of the disjunction of $\epsilon(v, \bar{i})$ for $1 \leq i \leq \min(r, k-1)$, together with $\epsilon(v, \overline{k+1})$ when $r = k$. Empty disjunctions mean zero; all index comparisons here are metalevel comparisons between fixed integers, not assumed arithmetic order predicates. Start with $t_0(v) = 0$ and set

$$t_r(v) = D(\neg_B b_r(v), t_{r-1}(v)).$$

Then $h_k = t_k$: each true condition contributes one successor, and each false condition leaves the value unchanged. The construction uses exactly k external syntax steps and the already available finite Boolean gates. It is not asserted to provide a single source term with variable k.

Write $A_k(n) = F(n; v.h_k)$ and introduce the distinct marker numerals

$$q_0 = \overline{k+1}, \qquad q_j = \overline{k-j} \quad (1 \le j \le k).$$

Since $h_k(0) = \overline{k}$, the initial state is q_0. Direct finite computation gives $A_k(\overline{j}) = q_j$ for $0 \le j \le k$. The displayed update has no output q_0, and for $1 \le j \le k$ it satisfies

$$h_k(v) = q_j \quad \longleftrightarrow \quad v = q_{j-1}.$$

Quantifier-free induction therefore gives $A_k(n) = q_0 \leftrightarrow n = 0$. For $1 \le j < k$, an external induction on j, using numerical coverage and the already proved predecessor uniqueness for $\overline{j}$, gives

$$A_k(n) = q_j \quad \longleftrightarrow \quad n = \overline{j}.$$

For clarity, in the successor case $n = Sm$ the left side is equivalent to $A_k(m) = q_{j-1}$, hence to $m = \overline{j-1}$. The right side has exactly the same condition by the previous predecessor theorem. The zero case has two false sides.

Now $Sx = \overline{k}$ implies $h_k(A_k(x)) = A_k(\overline{k}) = q_k$, so $A_k(x) = q_{k-1}$ and $x = \overline{k-1}$. This proves the next predecessor theorem. To extend numeral distinctness to $k+2$, a putative equality $\overline{k+2} = \overline{j}$ with $1 \le j \le k$ reduces, by the newly available predecessor theorems, to $\overline{k+1} = \overline{j-1}$, already impossible. Equality with zero is excluded by successor nonzeroness, and equality with $k+1$ by the no-fixed-point lemma. This closes the external induction.

Consequently no element other than the intended previous numeral can map to any fixed standard positive numeral. This does not rule out collisions over nonstandard targets: replacing the fixed construction above by a uniform variable-index argument would require an additional theorem that has not been supplied.

A simpler predecessor when cancellation is retained. If successor injectivity is retained, as in the supplied complete self-seeded theory, (2) gives $p_*(S^2m) = Sm$. Define the total single-valued finite-case graph

$$P(n) = \begin{cases} 0, & n = 0 \text{ or } n = 1, \\ p_*(n), & n \neq 0 \text{ and } n \neq 1. \end{cases}$$

Its graph uses only quantifier-free source guards and source value terms. Finite common refinements therefore eliminate each fixed quantifier-free formula containing P, preserving the available induction rule. The zero equation is immediate. For $P(Sn) = n$, use the already derived coverage to split $n = 0$ or $n = Sm$. The first gives $P1 = 0$. In the second, S^2m is neither zero nor one, and the displayed equation gives $P(S^2m) = Sm = n$.

Thus only the values at zero and one need correction, rather than the three exceptional values in the supplied stopped-counter construction. Coverage is proved first, and the subsequent predecessor proof no longer needs a joint coverage/predecessor induction. The remaining open issue is precise: can the weaker theory prove $p_*(S^2m) = Sm$, or otherwise prove $Sx = Sy \to x = y$? The proof above establishes neither statement.

4 The exact logical strength visible at one native binder

This section returns to the complete initialized-I native calculus of Section 2; its selector, predecessor, and PR compiler have been established. Those resources have not been claimed for the weaker self-seeded F theory of the preceding section.

This observation locates a concrete boundary that a proposed simplification of the induction guard must respect. It is a theorem about the established arithmetic interpretation of the native syntax, not a non-bi-interpretability theorem for every conceivable unrelated interpretation.

Write $C(0,a,b) = a$ and $C(Sn,a,b) = b$ for the derived numerical selector, and $P(0) = 0$, $P(Sn) = n$ for the derived predecessor. All formulas below may have arbitrary fixed numerical parameters. A formula is called Σ_2^{qf} here if it has the form $\exists x\,\forall y\,\theta(x,y,\vec{p})$ with quantifier-free θ. Several variables in a block may be packaged by the already derived primitive-recursive pairing operations. This notation specifies an actual prenex class; it does not appeal to classical prenex normalization of an arbitrary intuitionistic formula.

Proposition 4.1 (One-binder expressivity)**.** *Under the whole-domain interpretation in iPRA, the class of native formulas $D_{x,z}E(x,z,\vec{p})$ with E a literal equation is exactly the class Σ_2^{qf}, up to provable equivalence. Similarly, native $Q_zE(z,\vec{p})$ formulas are exactly the corresponding Π_1^{qf} class, up to provable equivalence.*

Proof. The forward direction follows from

$$D_{x,z}E \equiv \exists x\big(E(x,0) \to \forall z\, E(x,z)\big) \leftrightarrow \exists x\,\forall z\big(E(x,0) \to E(x,z)\big).$$

The final matrix is quantifier-free. The displayed interchange of implication and universal quantification is intuitionistically valid, because z is absent from $E(x,0)$. Dropping the outer existential gives the analogous statement for Q.

Conversely, choose the iPRA characteristic term $r(x,y,\vec{p})$ for a given quantifier-free matrix, with $r(x,y,\vec{p}) = 0 \leftrightarrow \theta(x,y,\vec{p})$. Expand C, P, and r into their fixed compiled initialized-iterator source terms, so the following formula is literally an equation of the native language. Define the literal equation

$$E(x,z,\vec{p}) : \quad C\big(z,0,r(x,Pz,\vec{p})\big) = 0.$$

Its zero instance is provable. Its successor instance is equivalent to $r(x,y,\vec{p}) = 0$, since $P(Sy) = y$. Numerical coverage therefore proves

$$\forall z\, E(x,z,\vec{p}) \leftrightarrow \forall y\, \theta(x,y,\vec{p}).$$

Since $E(x,0,\vec{p})$ is provable, its occurrence as the antecedent in the native D reading disappears by intuitionistic reasoning. Thus $D_{x,z}E$ has precisely the required existential-universal meaning. The same construction with no existential coordinate establishes the Q claim. □

Corollary 4.2 (A one-binder progress extension already adds a full prenex induction class)**.** *Suppose the native equation-progress introduction rule is extended to matrices $B(n,\vec{p})$ that consist of one D binder over a literal equation. Over the existing native theory, this extension derives every induction instance for Σ_2^{qf} formulas. Extending the rule to one Q binder over an equation derives the analogous Π_1^{qf} induction instances.*

Proof. Let $B(n,\vec{p})$ be the native translation supplied by the proposition. Suppose $\Gamma \vdash B(0,\vec{p})$ and $\Gamma, B(z,\vec{p}) \vdash B(Sz,\vec{p})$, with the usual fresh z. Apply the extended progress rule using an unused existential coordinate. It yields $D_{u,n}B(n,\vec{p})$. Paired elimination exposes $Q_nB(n,\vec{p})$, and closure elimination with the given base instance proves $B(t,\vec{p})$ for an arbitrary term t. The internal equivalence from the proposition transports this derivation to the original prenex formula. The argument for Q matrices is identical. □

Consequently, retaining a test that merely says "at most one binder" does not preserve the stated literal-equation induction restriction. Such a change supplies induction for this additional syntactic class; equivalence with the original rule would require a separate elimination or conservativity theorem. No claim here identifies every prenex induction extension with full HA, and no strength separation for arbitrary alternative bi-interpretations is inferred from this calculation.

4.1 A uniform lower bound on what larger depths can express

The one-binder equivalence extends to a useful inclusion at all depths. The qualification is important: no intuitionistic prenex-normal-form theorem is being silently used for nested native implications. Let $\delta(E) = 0$ for equations and $\delta(Q_y A) = \delta(D_{x,y} A) = 1 + \delta(A)$.

For a fresh numerical parameter z, define $G_z A$ by replacing every equation $s = t$ in the fixed finite formula A by

$$C(z, 0, s) = C(z, 0, t),$$

and retaining its native binders. Capture is avoided by renaming the bound variables first. This transformation does not change δ. It has the internal properties

$$\vdash G_0 A, \qquad G_{Su} A \leftrightarrow A. \tag{Gate}$$

For equations these are computation identities. For a Q or D root, the first assertion follows inductively because its entire matrix is true, using the zero witness for D; the second follows from the same interpreted binder applied to equivalent matrices. This reasoning uses the already recovered full logical interface, rather than assuming a new matrix-congruence rule during an earlier arithmetic bootstrap.

Theorem 4.3 (Alternating-prefix encoding at depth k)**.** *For every $k \geq 0$, every explicitly prenex formula*

$$\exists x_1 \forall y_1 \cdots \exists x_k \forall y_k \, \theta(\vec{x}, \vec{y}, \vec{p}),$$

with quantifier-free θ, has an iPRA-provably equivalent native formula of depth at most k. The translation is uniform by external recursion on the displayed finite prefix. In particular, allowing equation-progress introduction on all matrices of depth at most k derives induction for this entire prenex class.

Proof. For $k = 0$, replace the quantifier-free formula by its primitive-recursive characteristic equation. For the induction step suppose $F(x, y, \vec{p})$ already translates the remaining prefix at depth at most $k - 1$. Put

$$\mathcal{B}(F) := D_{x,z} \, G_z F(x, Pz, \vec{p}).$$

By (Gate), its zero matrix is true and its successor matrix at $z = Sy$ is equivalent to $F(x, y, \vec{p})$. Numerical coverage then gives

$$\mathcal{B}(F) \leftrightarrow \exists x \, \forall y \, F(x, y, \vec{p}).$$

Its depth is at most k, establishing the recursive construction. For induction, repeat the derivation in the preceding corollary with this translated matrix. The equivalences commute with the passive induction parameter and its zero and successor substitutions. □

Blocks of several numerical variables can be packaged into one tuple using the available pairing and projections. Dummy blocks can be added because the numerical domain is inhabited. Consequently the inclusion covers the usual explicitly prenex Σ_{2k} class, and for $k \geq 1$ the Π_{2k-1} class by an

unused first existential block. It does not claim that every intuitionistic formula of a conventional arithmetical hierarchy has been prenexed without additional assumptions.

At depths greater than one, only this encoding inclusion is claimed intuitionistically. The native reading contains implications whose antecedents may themselves be quantified; moving such quantifiers through implication is not a general intuitionistic equivalence. Under classical logic the usual prenex manipulations give the familiar Σ_{2k} upper bound for a D root of depth k, but that classical observation must not be substituted for an iPRA proof. The new inclusion and the exact depth-one calculation already suffice to audit the literal-equation guard.

5 The complete proof kernels and their invariants

The recommended complete program is printed here. It imports only the standard JSON parser. The theorem producer, tests, arithmetic metatheory, and corpus audit are not trusted imports.

```
import json

def c(a, g=[[]], k=0):
    i = lambda a, t=None, d=0: [i(b, t, d + 5 - len(a)) for b in a] if isinstance(a, list) else a if k <= a <
      d else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    c(m, g, k)
    a = m in ([s, u], not s and [t] * 3) and i(t, u) == a and i(t, s) or a
    h = m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

The more compact list test in `code/checker_bytes.py` replaces `isinstance(a, list)` by `a * 0 == []`. For JSON values, only arrays satisfy that test; unsupported multiplication or a failed numerical comparison rejects the input. The explicit test is retained in the recommended program for readability. A fully expanded standalone version is supplied as `code/readable_checker.py`.

5.1 Public contract and sorted expression encoding

Input is one finite JSON array whose last element is the goal. Earlier four-field packets $[m, s, u, t]$ are consumed in reverse order. Acceptance requires normal process exit and exactly `ACCEPT` followed by a newline. Exceptions, nonzero exits, resource failures, and other output are rejection. Unused certificate prefixes are permitted. Completeness is the usual adequate-resource claim about finite canonical proofs; there is no assertion that fixed hardware checks arbitrarily large proofs.

The inherited representation is sorted: the numerical and formula interpretations of an array differ.

Arity	Numerical interpretation	Formula interpretation
0	Zero	The theorem $0 = 0$
1	Successor	Paired binder D
2	Zero; annotations ignored	Equality
3	Scalar iterator I	Closure Q, last child its matrix
≥ 4	Zero; annotations ignored	The theorem $0 = 0$

Canonical forms include $[n]$ for successor, $[n, a, h]$ for iteration, $[s, t]$ for an equation, $[B]$ for D, and $[B, B, B]$ for Q. Formula root depth is zero and numerical root depth is three. Under p protected numerical slots the depths are $2p$ and $3 + 2p$. Coordinates $0, 1, 2$ denote numerical zero; coordinates $3 + 2j, 4 + 2j$ name protected slot j. Admitted negative labels name rigid parameters.

The common child-depth increment is $5 - r$ at an array of arity r. A successor child is read beneath two dummy zero slots. The counter and initializer of an iterator are beneath one dummy zero slot; its update is beneath the actual state slot. The same increment therefore implements the different binding conventions. A scalar escaping a protected numerical environment is the substitution hole, whose image is checked at numerical root depth three. Formula scalar padding has the inherited truth meaning. This is a representation of finite terms, not a new arithmetic operation or a total evaluator for numerical codes.

5.2 Lexical scopes and the crucial allocation order

At a call $c(a, g, k)$, the opener closes over that call's fresh-name floor k. A major subproof receives the same context and floor value. Its private allocation cannot change the caller's floor. Context extension constructs a new list in the recursive programs. In the loop profile, the context is an immutable tuple and `+=` rebinds it. The default list in the recursive programs is never mutated.

The current goal is checked at the old floor. Raw hypothesis construction and witness opening occur before decrementing k; every recursive-image or witness check triggered there uses that old floor. The final hypothesis opening may admit fresh names, discharged by the zero-instance argument below. Different completed major branches may reuse names because their assumptions cannot escape. A finite raw certificate may use fresh fractional numerical labels; these are treated as a finite family of fresh parameters, as in the inherited raw-input proof. Canonical certificates use only integer coordinates.

The order is essential. The delivered four-packet attack in `tests/future_witness_capture.json` targets

$$D_{x,y}\big[I(y, 0; v.x) = I(y, 0; v.y)\big].$$

Its zero equation is true. At every successor y, its equation says $x = y$. Thus its ordinary meaning requires a single number equal to every positive number, which is false already at 1 and 2. An unsafe mutation that allocates the new name before opening the witness accepts this formula: the witness can then be chosen as the successor of the induction eigenparameter. The corrected programs reject it. This is a concrete reason to retain the construction-before- allocation order, not merely a formatting preference.

5.3 Neutral major premises and folded branches

The initial hypothesis `[]` denotes the already derived theorem $0 = 0$. Every non-hypothesis command checks its major m unconditionally. A canonical command without a substantive major supplies `[]`; its proof closes immediately and allocates no name. The inherited list-valued-context invariant excludes numeric scalar majors from a finite successful checking trace. In particular, scalar zero cannot reach the matcher as a false-valued major. This is why the potentially dangerous Python `False` alternative in the matcher is harmless. The complete raw argument is reproduced in the final appendix.

For a surviving nonempty list m, the list $m * 3$ is nonempty. For a computation command, the constructed hypothesis is a two-element list, also nonempty. Consequently the original three-way

choice of major export, computation, and introduction is equivalently implemented by

```
m*3 or (u and COMPUTATION) or INTRODUCTION.
```

No semantic branch is removed. The AST profile folds only the outer major choice. Its loop replaces a tail recursive continuation with a context-preserving iteration; it does not change an inference rule.

In an introduction, both m and u are false-valued. The mode comparison is with u: zero selects progress for a literal equation, while an empty array selects the zero case. No false-valued JSON mode equals the odd arity of a meaningful compound matrix. Replacing u by the unrestricted source field s would permit compound induction; the archive contains a targeted forbidden-induction certificate for that error.

These changes are compositions of documented existing techniques. Their particular complete source files were absent from the frozen corpus, but no foundational novelty is inferred from that fact.

5.4 Computation supply and missing literal-zero cases

The direct computation interface uses truthy syntactic initializers, and a truthy predecessor for a successor instance. This is not a numerical nonzeroness test. The finite term

$$Z_* := I(S(S0), S0; v.0)$$

has a directly permitted proof of $Z_* = 0$. For an excluded computation, replace its offending zero predecessor and initializer positions simultaneously by a fresh placeholder in a fixed equation $E(w)$. Then $E(Z_*)$ is directly permitted; transport along $Z_* = 0$ gives $E(0)$. One transport and its two computation premises repair the leaf. Thus unrestricted I0 and IS are recovered without an arithmetic oracle or an extra axiom.

A generated raw hypothesis may contain newly admitted names and need not be true uniformly in those names. The sound rule is

$$\vdash H(\vec{0}), \qquad \Gamma, H(\vec{b}) \vdash A \quad \Longrightarrow \quad \Gamma \vdash A, \qquad \vec{b} \notin \mathrm{FV}(\Gamma, A).$$

Substitute zero for all the finitely many new names in the continuation, rename later eigenparameters first, and cut in the proved zero instance. The old-floor validation is exactly what licenses this discharge.

5.5 Soundness and canonical completeness

Induct on a finite successful trace. Hypothesis closure uses a retained formula. A recursive major is proved in the caller's old context. Endpoint matching realizes transport or closure elimination. A unary paired major exports its closure at a fresh witness; other surviving arities export only truth padding. Introduction retains exactly the permitted case or literal-equation progress hypothesis. Computation supply uses the preceding zero-instance discharge argument. Finally cut away the initial derived truth. This yields a derivation in the native calculus of Section 2.

Conversely, start with a finite native derivation, normalize the excluded computation leaves as above, and alpha-rename its parameters with a branch-local name supply. Major subproofs start at the caller's floor, and continuations allocate their own next name. Canonical absent majors

are empty arrays. Folding the nonempty branches chooses the same hypothesis, and looping the tail continuation has the same effect. The resulting finite certificate is accepted. The decoded canonical theorems are therefore exactly those of the complete native calculus; its full-domain iPRA interpretations transfer unchanged.

This argument does not claim identical accepted languages for all redundant malformed JSON spellings. In particular, the generic opener admits some iterable truth aliases, while folded multiplication may reject them before a continuation. That only removes redundant raw encodings and leaves canonical completeness intact.

5.6 The generic profile's qualification

The generic AST profile attempts iterable traversal and catches failure. A nonempty malformed string can cause repeated traversal of its own one-character strings. Real CPython ultimately rejects through its stack limit; an ideal unlimited-stack interpreter can diverge. Thus 289 AST nodes is not claimed as the size of a total abstract parser on all malformed JSON. The recommended explicit-list program and the compact-list variant recurse structurally through finite trees and reject unsupported leaf types. The arithmetic and metatheoretic claims use finite successful traces and finite canonical proofs.

The written proofs have not been formalized in Lean or Rocq. The regression suite tests the implementation and concrete counterexamples; it does not prove the universal soundness or bi-interpretability theorem.

6 Evidence, reproducibility, and limitations

All recorded Python measurements use CPython 3.12.14 and zlib 1.3.2. The measurer removes genuine docstrings, applies `ast.unparse`, and appends a newline. It then counts UTF-8 bytes, significant Python tokens, and every AST node of the whole standalone module, including JSON input and the verdict driver. The common Python/JSON runtime is excluded, matching the inspected drafts' convention. There is no hidden theory-specific import, embedded binary theorem engine, or decompression trick in a selected checker.

Gzip is reported with `compresslevel=9` and `mtime=0`, including its wrapper. It is an informational metric. Raw and normalized gzip lengths are both retained. For the three normalized compact sources these lengths coincide. The readable expansion has 2,307 raw bytes, 1,463 normalized bytes, 295 tokens, and 348 AST nodes; its raw gzip size is 961 bytes and its normalized gzip size is 546 bytes.

The checker suite has 49 local certificate cases and 46 global-reference cases. Together with malformed-input cases it checks 756 complete-process expectations across the four new programs and three frozen baselines, under ordinary and optimized Python. Six additional expectations run three deliberately invalid implementation variants in both modes. All pass with zero mismatches. The mutations exercise compound-progress selection, context leakage from a subproof, and future-variable capture by premature name allocation. Their targeted certificates are rejected by the corrected programs and accepted by the corresponding mutants.

The independent arithmetic diagnostics evaluate the displayed source Boolean operations and protected orbits. They check 361 orbit/parameter pairs, 19 right-inverse inputs, 21 coverage/predecessor inputs, 21 fixed-point/marker inputs, six fixed-prefix indices, and 63 finite-update source

cases. Seven targeted bad controls detect missing zero protection, missing stopping, omitted finite exceptions, and corrupted marker operations. A separate branching-successor example illustrates the invalidity of cancelling S from a right-inverse identity alone. It is explicitly *not* a model of all the weak theory's recursor and induction schemas.

Two independent assistant workers reviewed the coverage and fixed-prefix arguments, with the detailed logical review retained in the archive. This is internal independent checking, not external human peer review or proof-assistant certification. The tests do not establish a theorem about nonstandard elements; that conclusion rests on the written fixed-parameter induction arguments.

6.1 Reproduction commands and files

From the extracted archive directory, run

```
python3 verify_manifest.py
python3 reproduce.py
python3 build.py
```

The first verifies distributed file identities. The second remeasures the complete modules and frozen corpus, regenerates the certificates, checks the process expectations, and runs the arithmetic diagnostics. The third builds the standalone TeX source and PDF with shell escape disabled. The tests require only Python's standard library. Building the report requires a standard pdfLaTeX installation.

`evidence/snapshot.json` fixes the source commit and archive blob identities. `evidence/corpus_metrics.json` records all 484 distinct source measurements and locations; `evidence/boundary_classification.json` records the 48 relevant role classifications. Exact frozen Python sources are supplied under `evidence/source_corpus/`, so the static remeasurement needs no network. To repeat the full archive-identity audit against a local checkout of the pinned repository, run

```
python3 audit_corpus.py /path/to/ProveIt/docs/drafts
```

No inspected corpus program is executed by that audit. Only the selected and explicitly attributed test programs are executed during reproduction.

The complete compact and readable checkers are under `code/`; the four-packet witness-capture attack and regenerated named proofs are under `tests/`. `reference/` retains the measured incumbents, the original inherited mathematical sources, and the upstream MIT-0 license. Attribution and content hashes are recorded in `evidence/mathematical_provenance.json` and the source manifest. The adapted arithmetic appendix makes its scalar prerequisites explicit before general PR compilation, avoiding a possible circular reading of the original exposition.

The complete deliverable supports the specific new lemmas, small source reductions, and inherited full-iPRA interpretation stated here. It does not establish a single all-coordinate record, an absolute minimum, or an injectivity-free complete self-seeded axiomatization.

7 Further research questions

1. Can the weak self-seeded theory prove successor injectivity? The proved identity is $S(p_*(S^2m)) = S^2m$. A derivation of $p_*(S^2m) = Sm$ would close a central gap; simply cancelling the outer S assumes exactly what is missing.

2. Can the fixed-standard-index marker construction be internalized uniformly without assuming a general primitive-recursive compiler? Every fixed numeral target has a unique predecessor, but passing from that external family to one variable-index theorem is not automatic.
3. If cancellation is not derivable, can one construct a model of *all* staged recursor equations and equation-induction instances with a successor collision? A branching successor graph alone does not meet those obligations.
4. Can a single complete, structurally terminating checker beat the current 261-token minimum and the new 531-byte and 289-node results simultaneously? Merely mixing coordinates of different programs does not answer this question.
5. Can a new certificate invariant remove a whole rule or validation operation while preserving the entire consequence relation? The concrete scope and compound-induction attacks in this archive are useful regression targets for such a proposal.
6. Can the new coverage, standard-target, and binder-depth proofs be formalized in Lean or Rocq using only the exact source schemas? Numerical testing and an informal stronger induction principle would not constitute that formalization.

A The ordinary intuitionistic logic used in the arithmetic arguments

The native calculus has exactly the primitive rules in Section 2; it derives the following interface. The weaker self-seeded theory in Section 3 instead uses this ordinary interface explicitly. It is therefore charged as part of that theory, not treated as free hidden machinery.

Formulas are built from numerical equations and $\bot$ with $\wedge, \vee, \to, \forall, \exists$. The rules are single-conclusion intuitionistic natural deduction:

$$\frac{A \quad B}{A \wedge B} \quad \frac{A \wedge B}{A} \quad \frac{A \wedge B}{B} \qquad \frac{A}{A \vee B} \quad \frac{B}{A \vee B} \quad \frac{A \vee B \quad [A] \vdots C \quad [B] \vdots C}{C},$$

$$\frac{[A] \vdots B}{A \to B} \qquad \frac{A \to B \quad A}{B} \qquad \frac{\bot}{A},$$

$$\frac{A(a)}{\forall x A(x)} \quad \frac{\forall x A(x)}{A(t)} \qquad \frac{A(t)}{\exists x A(x)} \quad \frac{\exists x A(x) \quad [A(a)] \vdots B}{B}.$$

In universal introduction, a is fresh for the undischarged context. In existential elimination, a is fresh for that context, the conclusion B, and the passive parameters of the quantified formula. Brackets mark the hypotheses discharged by the inference. There is the hypothesis rule and the usual reusable-context structural behavior. Equality has reflexivity $t = t$ and capture-free transport from $s = t, A(s)$ to $A(t)$ for every fixed formula template. These imply the usual symmetry, transitivity, and function congruence laws. Negation abbreviates $A \to \bot$; biconditional abbreviates the conjunction of both implications. Neither excluded middle for arbitrary formulas nor double-negation elimination is a rule.

For reference, the contextual literal-equation induction schema is

$$\frac{\Gamma \vdash s(0, \vec{p}) = t(0, \vec{p}) \qquad \Gamma, s(z, \vec{p}) = t(z, \vec{p}) \vdash s(Sz, \vec{p}) = t(Sz, \vec{p})}{\Gamma \vdash s(a, \vec{p}) = t(a, \vec{p})}.$$

The parameter z is fresh for the retained context, target, and fixed parameters. This restriction is part of the schema. The derived quantifier-free induction used in the new coverage proof is obtained by translating each fixed quantifier-free formula into its Boolean characteristic equation, rather than by silently allowing a larger primitive induction rule.

B Reconstructing the intuitionistic logical interface

This appendix reproduces the relevant part of the supplied Choice-DI development, with its Q-context form used in the main text. It is included to make the inherited logical interpretation explicit.[1]

B.1 Induction and numerical cases separate cleanly

Lemma B.1 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.2 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.2 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v,a,b) := I(v,a;u.b)$ with u unused. Computation gives $C(0,a,b) = a$ and $C(Sv,a,b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s=t) := (C(v,0,s) = C(v,0,t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u,y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s=t,u=w) := (C(v,s,u) = C(v,t,w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A,\mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A,B),\\ M_v(E,\mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE,B),\\ M_v(\mathsf{D}_{x,y}A,E) &:= \mathsf{D}_{x,y}M_v(A,G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A,B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A,B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A,B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0=1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A,B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\,(x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.3 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External

induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The arithmetic bridge in the following appendices identifies T_{eq} with full iPRA; the arithmetic appendices below establish that bridge for the complete native rule inventory stated in the main text.

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [7] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{4}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{5}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{6}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{7}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{8}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (8) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{9}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (8); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{10}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (9), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{11}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{12}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (12). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (13) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (14) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (15)$$

The base is (14). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (16)$$

The first uses the comparator recurrence. The second uses (15) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (17)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (18)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (17) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [8]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language here is the staged scalar-iterator language of the main text. All operations in this appendix are the finite scalar terms listed in its opening paragraph. Only their already proved equations are used; general primitive recursion is compiled in the following appendix, after QFI. In particular, predecessor and selector satisfy

$$P(0) = 0, \qquad P(Sx) = x, \tag{19}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{20}$$

Write $1 = S0$. Besides those scalar equations, use

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{21}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{22}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{23}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (23) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{24}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{25}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{26}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (19). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (25); symmetry and (22) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{27}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{28}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{29}$$

To prove the range assertion, split (21). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (20) also give, for all u, v,

$$C(u,v,v) = v. \tag{30}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{31}$$

If $u = 1 = S0$, (26) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{32}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{33}$$

Using these equations and (23), we now derive

$$0 + y = y, \tag{34}$$
$$(Sx) + y = S(x + y), \tag{35}$$
$$d(Sx, Sy) = d(x, y), \tag{36}$$
$$d(Sa, a) = 1. \tag{37}$$

Each induction below has a single equation as its induction formula.

For (34), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (35), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (35) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{38}$$

Indeed, both sides are $S(y + r)$.

For (36), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (37). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (36) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{39}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (34) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (30).

For the step, assume (39) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (39) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (38) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (39). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{40}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (39). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (37) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{41}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{42}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{43}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (28), which is equivalent to $d(Sa,b) = 1$ by (31), and then to $a = b$ by (40). The range assertion is (27) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (42) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, (a = b \vee (a = b \to \bot)) \tag{44}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{45}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{46}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{47}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{48}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{49}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (42)–(43). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (49) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (49), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [9]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

F Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u,\vec{p})$, introduce $i_h(n,a,\vec{p})$ with the two equations

$$i_h(0,a,\vec{p}) = a, \qquad i_h(Sn,a,\vec{p}) = h(i_h(n,a,\vec{p}),\vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

F.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem F.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

G Attribution of the raw-input audit

The following raw-input analysis is reproduced from the neutral-major draft at the pinned snapshot.[3] Its occurrences of "new" refer to that earlier protocol. Section 5 proves the additional lexical-scope and branch-folding changes used here. The minimal-successful-trace argument below quantifies over arbitrary starting floors, so it continues to apply when major proofs allocate only local names. The original unmodified source is preserved in `reference/`.

H Why the absent major can be a proved empty array

This section audits the new control-flow change on raw input, including noncanonical JSON. It uses the inherited sorted arity decoder and semantic-opening lemmas for the numerical and logical rules. It does not replace those lemmas with finite testing or claim proof-assistant certification.

H.1 The exact change

The initial context is the one-element list `[[]]`. Its sole formula, the empty array, denotes $0 = 0$ under the inherited decoder. At every non-hypothesis command the checker now executes

```
m, s, u, t = q()
c(m, g)
```

without testing whether `m` is nonempty. A canonical command with no substantive major puts `[]` in this field. Its premise closes immediately by hypothesis, without consuming a command or changing the fresh-name floor.

The final choice of the retained assumption uses

```
(m or len(a) % 2 == u) and k
```

instead of comparing the arity parity with `m`. In an introduction command, `u=0` selects equation progress and `u=[]` selects the zero-case premise. This reuse is legitimate because introduction is reached only when both `m` and `u` are false-valued. Computation still requires a truthy initializer `u`; its missing literal-zero cases have the inherited certified-zero repair.

The source is deliberately left with the old matcher

```
m in ([s, u], not s and [t] * 3)
```

and no added `m and` guard. As an isolated Python expression this would be unsafe: if `s` is truthy, its second alternative is `False`, which compares equal to numeric zero. The following whole-program invariant proves that this dangerous major cannot reach the matcher.

H.2 Two facts about the root opener

Call a Python value *numeric* here when it is an integer, a float, or a Boolean returned by the JSON loader. The finite Python execution also admits the loader's optional nonfinite float values. No custom numeric class is supplied by a certificate.

Write $O_k(r;t)$ for the opener on the numeric value r, with image t and its default formula-root depth zero. Direct evaluation of the program gives

$$O_k(r;t) = \begin{cases} r, & k \le r < 0, \\ 0, & \text{otherwise.} \end{cases} \qquad \text{(Root scalar)}$$

In particular this result is numeric and independent of the image t. The formula covers negative allocated names and negative fractional names without identifying different numerical values. Signed zero and Booleans have Python's ordinary numerical equality behavior. A NaN fails the interval test and returns zero; the relevant infinite values also return zero. None of these results is a list.

A second fact is that a successful root opening of a list returns a list. Under either strict list predicate, no other input type can return a numeric result except a numeric input. Under the generic iterable opener, a successful iterable traversal also returns a list; if traversal fails, the numerical comparison cannot succeed on a list, string, dictionary, or null. Thus in every profile:

> If a root opening returns a numeric result, its argument was numeric; if it succeeds on a list, its result is a list.

These statements concern finite successful evaluations. They assert neither arbitrary revalidation idempotence at ignored annotations nor unlimited-stack termination for the generic iterable profile.

H.3 No numeric goal can be proved from list-valued assumptions

Lemma. For any initial integer fresh-name floor and any finite remaining certificate, there is no finite successful invocation `c(a,g)` whose raw goal `a` is numeric and whose every context element is a list.

Proof. Suppose otherwise, and select such a successful invocation with the fewest calls to `c` in its completed recursive trace. Its first hypothesis check cannot close: the root opening of its numeric goal is numeric, and no numerical JSON value equals a Python list.

The invocation must therefore consume a command and successfully return from `c(m,g)`. If `m` were numeric, this would be a strictly shorter successful invocation of the forbidden kind. Thus the major is not numeric.

In either strict profile, a successfully checked nonnumeric major is a list. In the generic profile an empty string or an empty dictionary can also open to the empty list. A nonempty dictionary can occasionally have its iterable alias proved, but it cannot occur as a truthy major in the selected completed parent invocation: the subsequent expression `m * 3` raises a type error. A nonempty string has no finite successful opening. Consequently every truthy major that survives to the continuation exports a list, and every surviving false-valued major is an empty list or one of those harmless empty iterable aliases.

Consider the possible change of goal. If the matcher does not fire, the goal stays numeric. If it fires, its target test requires `i(t,u) == a`. The second root-opening fact therefore implies that `t` is numeric. By (Root scalar), `i(t,s)` and `i(t,u)` are the same root result. Both the Boolean-expression and the explicit-conditional implementations consequently retain a numeric goal. The Boolean implementation may retain the original raw zero alias when the transported result is false-valued; that makes no difference to this assertion.

If the major is truthy, the generated hypothesis is the list `m * 3`. If the major is false-valued and `u` is truthy, the computation expression constructs a two-element list. A successful opening of either hypothesis is a list. The final recursive call then has a numeric root-opened goal and an entirely list-valued context. It is a proper subtrace of the selected invocation, contradicting its minimality.

The only remaining case is introduction. Its expression `i(*a,t,2)` must unpack the numeric raw goal as an iterable, which raises a type error. That case cannot belong to a successful trace. All possibilities have been excluded. □

The minimization ranges over all starting floors and stacks. It is therefore unaffected by a recursively checked major decreasing the global floor before its parent resumes. The argument uses finite completed traces, not a presumption that recursive checking always terminates.

H.4 Every reached command has a list-valued context

Initially the only assumption is the list `[]`. Suppose a command is reached with list-valued assumptions and its major check returns. The lemma excludes a numeric major. A truthy surviving major exports a list; a computation constructs a list. In introduction the assignment is

```
h = a = i(*a, t, 2)
```

in mathematical order. If its result is numeric, the later `len(a)` raises a type error before the continuation context is constructed, since this branch has false-valued `m`. Such a scalar matrix does *not* insert a scalar-zero assumption. If the assignment yields a list, successful opening again yields a list.

Both recursive premises thus receive list-valued contexts whenever execution reaches them. The expression `g + [new]` constructs a new list; a premise cannot mutate its caller's context. Induction on the reached calls proves the invariant. The tuple-based explanatory variant has the corresponding immutability property.

It follows that no numeric zero, Boolean `False`, floating zero, truthy numeric scalar, or newly allocated numeric name can be a successfully verified major at a reached command. In particular a closed scalar-zero goal is rejected by this interface. The canonical empty-array truth is accepted

instead; mathematical completeness is claimed for canonical encodings, not for every redundant truth alias.

H.5 The unguarded matcher is safe on every reached continuation

If `s` is false-valued, both matcher alternatives are nonempty lists, so no false-valued major can match. If `s` is truthy, the second alternative is `False`. A false-valued major reaching this point is an empty list, or in the generic profile an empty string or dictionary. None equals `False`; a numeric major has already been excluded. Therefore, at every point reached after a successful major check, the matcher has exactly the same truth value as

```
m and m in ([s, u], not s and [t] * 3)
```

even though this identity is false on unrestricted Python tuples of values. No sentinel and no extra source guard are needed.

The proof depends on the initial context being `[[]]`, not `[0]`, and on premise contexts being freshly constructed. Silently changing either detail would invalidate this argument.

H.6 Why the relocated induction mode must be the initializer field

The corrected selector compares arity parity with `u`. In an introduction branch this field is necessarily false-valued. No false-valued JSON value compares equal to one. Thus a meaningful compound matrix, of odd arity one or three, cannot receive a progress assumption. A literal equation has even arity two and receives progress precisely for a zero-like mode such as `0`; an empty array mode chooses the zero instance. Surplus arities retain their inherited constant interpretations. Scalar matrices cannot survive the shape check discussed above.

Comparing instead with the unused `s` field would be wrong. That field is unrestricted in introduction, so choosing `s=1` would enable progress for odd-arity compound matrices. The distinction is semantic, not merely a preferred serialization.

In the computation branch `u` is truthy. Its new comparison can choose a fresh-name opening for some noncanonical scalar initializer aliases. This is handled by the inherited zero-instance discharge lemma, not by claiming that every generated open hypothesis is uniformly true. If the generated hypothesis is $H(\vec{b})$, its zero completion $H(\vec{0})$ is a computation theorem, while the old-floor validation makes the finite family of newly admitted names $\vec{b}$ fresh for the retained conclusion and context. Substituting zero for those names and cutting in the computation theorem discharges the extra hypothesis. Negative fractional names in the newly admitted interval are included separately in this finite substitution.

H.7 Conservativity and canonical completeness

The initial empty-array formula is already a theorem. For example the permitted nonempty-seed computation gives

$$C = 0, \qquad C := I(S(S0), S0; v.0).$$

Use this equation as both the equality major and the premise of transport through $F(w) \equiv (w = 0)$ to obtain $0 = 0$. Adding that fixed theorem to a proof context is conservative by substitution of its finite derivation for hypothesis uses. No new object-level truth predicate or logical axiom has been introduced.

For canonical certificates, substantive major commands retain their old fields. A computation uses the absent-major field `[]` and its existing nonempty initializer. An introduction uses absent major `[]`, with `u=0` for equation progress and `u=[]` for cases; its witness remains in `t`. The added empty major closes without a command or name allocation, so the remaining canonical proof simulation is unchanged. First applying the existing certified-zero repair supplies every unrestricted computation instance. This gives a finite certificate for every canonical derivation of the inherited native calculus.

Conversely, the invariant makes each reached matcher a legitimate guarded matcher. The remaining commands use the inherited transport, closure, paired-introduction/elimination, and zero-instance computation arguments, with the corrected equation-only mode test. Discharging the initial $0 = 0$ theorem yields a derivation in that same calculus. Its established whole-domain iPRA interpretations therefore transfer without changing the numerical domain, equality, or object language. The new result concerns a proof-checker presentation, not a new primitive-recursive arithmetic foundation.

H.8 Raw JSON types and the generic profile

The strict profiles recognize only Python lists as containers. The short predicate `a * 0 == []` has this effect for JSON values: lists pass; strings and numbers give different zero products; dictionaries and null raise. Its numerical fallback rejects strings. The root invariant covers all numbers and Booleans that reach it, including nonfinite values accepted by the default Python loader.

The generic profile has the inherited additional aliases. An empty string and an empty dictionary open to `[]`; a dictionary with only the empty-string key opens to `[[]]`, with its value ignored. A truthy dictionary cannot survive export as a major because `m * 3` raises. Nonempty strings and nonempty dictionary keys have no finite successful opening. Actual CPython rejects them through its recursion limit and subsequent type failure, while an ideal unlimited-stack execution can diverge. The raw soundness argument above covers every finite successful run of this profile; it does not assert ideal termination on every malformed finite JSON input.

Larger arrays remain arrays throughout opening, irrespective of their ignored annotations or depth increments. They therefore do not threaten the scalar-versus-list invariant. Their formula meanings and freshness behavior remain subject to the inherited sorted-decoder proof. Physical time, memory, and stack limits can reject sufficiently large genuine proofs in every profile; the completeness claim has the same adequate-resource, canonical-proof interpretation as its predecessors.

References

[1] Vladimir Reshetnikov, *ProveIt*, frozen draft corpus at commit `b9c4d74a0018bdafc5bc06e755a0145f4ed2b8d6`. https://github.com/VladimirReshetnikov/ProveIt/tree/b9c4d74a0018bdafc5bc06e755a0145f4ed2b8d6/docs/drafts. Retrieved and byte-verified 10 November 2026.

[2] Vladimir Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, maintained project README, commit `89f5bdfe`, 10 November 2026. https://github.com/VladimirReshetnikov/ProveIt/blob/89f5bdfef815cd2b2c5422978896edc9c819af3f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md.

[3] *Equality from monus and neutral major premises*, `ipra_monus_and_neutral_major_2026-11-10.zip`, in the pinned corpus. Source of the neutral-major protocol and its raw-input audit.

[4] *Scalar self-seeded arithmetic and the iPRA checker frontier*, `ipra_selfseed_scalar_2026-11-10.zip`, in the pinned corpus. Source of the complete self-seeded presentation, lexical profiles, and the collected inherited arithmetic appendices.

[5] *Arity calculus for iPRA*, `ipra_arity_calculus_2026-11-10.zip`, in the pinned corpus.

[6] *Certified zero*, `ipra_certified_zero_2026-11-10.zip`, in the pinned corpus.

[7] Inherited scalar-predecessor development, `reference/scalar_predecessor.tex` in the self-seeded archive of reference [4]. The exact source hash is recorded in `evidence/mathematical_provenance.json`.

[8] Inherited equality-reflection development, `reference/equality_reflection.tex` in reference [4]. The presentation here explicitly restricts its function inventory to the already constructed scalar terms before general PR compilation.

[9] Inherited pairing and primitive-recursion development, `reference/primitive_recursion.tex` in reference [4].